

Nania Vathana

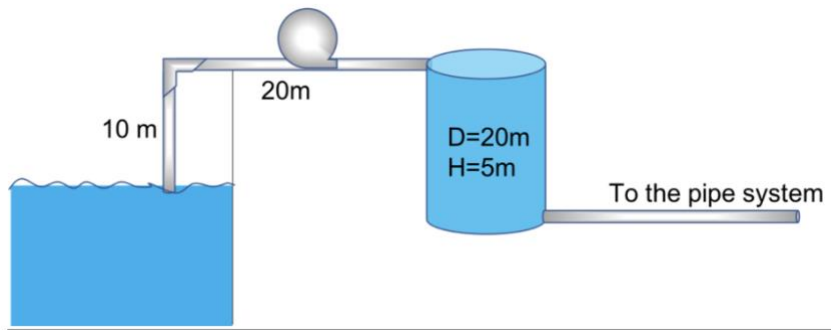
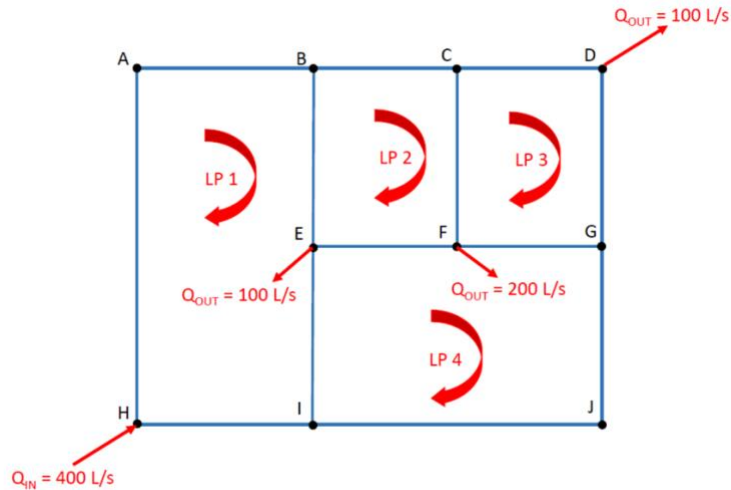
R11954346

ChE 3315 Fluid Mechanics

Hardy Cross Pipe Network Analysis (Due May 8th, 2026)

1. Problem statement

A pipe network consisting of one inlet, three outlets, and four flow loops is analyzed using the Hardy Cross iterative method. The network is supplied by an open tank at pipe level, water is pumped from a reservoir 10 m below the pipe network into the tank. The system is described in the diagram below.



1.1 The network contains 10 nodes arranged in three rows and 13 pipe segments

- Top row nodes: A, B, C, D
- Middle row nodes: E, F, G
- Bottom row nodes: H, I, J
- Inlet: Node H, $Q_{in} = 400 \text{ L/s}$
- Outlets: Node D (100 L/s), Node E (100 L/s), Node F (200 L/s)

1.2 Pipe Data and Initial Flow Guesses

The table below lists the friction factor, length, diameter, and initial flow guess for all 13 pipe sections. Initial flow guesses must satisfy mass conservation (continuity) at every node.

Pipe	f	L (m)	D (mm)	Qi (L/s)	Assumed Direction
A-B	0.0223	150	225	200	A→B
B-E	0.0175	125	200	100	B→E
E-I	0.0223	125	225	0	—
I-H	0.0218	150	300	200	H→I
H-A	0.0218	250	300	200	H→A
B-C	0.0233	75	225	100	B→C
C-F	0.0175	125	200	0	—
F-E	0.0188	75	150	0	—
C-D	0.0233	75	225	100	C→D
D-G	0.0175	125	200	0	—
G-F	0.0188	75	150	200	G→F
G-J	0.0233	125	225	200	J→G
J-I	0.0218	150	300	200	I→J

1.3 Initial Mass Balance Verification

For the initial guesses to be valid, the continuity equation must be satisfied at every node: the algebraic sum of all flow rates entering and leaving each node equals zero (accounting for external inflows and outflows).

Node	Pipes Connected	External Flow (L/s)	Net Balance (L/s)
A	A-B (out), H-A (in)	0	$-200 + 200 = 0$
B	A-B (in), B-E (out), B-C (out)	0	$200 - 100 - 100 = 0$
C	B-C (in), C-F (out), C-D (out)	0	$100 - 0 - 100 = 0$
D	C-D (in), D-G (out)	-100 (outlet)	$100 - 0 - 100 = 0$
E	B-E (in), F-E (in), E-I (out)	-100 (outlet)	$100 + 0 - 0 - 100 = 0$
F	C-F (in), G-F (in), F-E (out)	-200 (outlet)	$0 + 200 - 0 - 200 = 0$
G	D-G (in), G-F (out), G-J(out)	0	$0 - 200 - (-200) = 0$
H	H-A (out), I-H (in)	+400 (inlet)	$400 - 200 + (-200) = 0$
I	E-I (in), I-H (out), J-I (in)	0	$0 - (-200) + (-200) = 0$
J	G-J (in), J-I (out)	0	$(-200) - (-200) = 0$

2. Hardy Cross Method

The Hardy Cross method is an iterative process used to solve for flow rates in a closed-loop network. It relies on the principle that the sum of head losses around any closed loop must equal to zero.

2.1 Equations

- **Darcy-Weisbach Head Loss (h_L):**

$$h_L = K|Q|Q \quad \text{where} \quad \frac{8fL}{g\pi^2 D^5}$$

- **Hardy Cross Correction (ΔQ)**

$$\Delta Q = - \frac{\sum KQ_i|Q_i|}{\sum 2K|Q_i|}$$

- **Pump Power (P)**

$$P = \rho g Q h_p$$

3. Appendix

```
%% ChE 3315 Fluid Mechanics - Finals Project
% Hardy Cross Method for Pipe Network Analysis
% Due: May 8th, 2026
%
% Pipe Network: 1 inlet (H), 3 outlets (D: 100 L/s, E: 100 L/s, F: 200 L/s)
% Total inlet flow: Q_IN = 400 L/s at node H
% Four flow loops: LP1, LP2, LP3, LP4
%
% Nodes: A, B, C, D (top row)
%         E, F, G   (middle row)
%         H, I, J   (bottom row)
%
% Sign convention: Clockwise flow in a loop is POSITIVE

clc; clear; close all;
tic; % Start timer - records total code runtime

%% =====
% SECTION 1: PIPE DATA
% =====
% 13 pipe sections given in problem table
% Pipe index:
% 1:A-B  2:B-E  3:E-I  4:I-H  5:H-A
% 6:B-C  7:C-F  8:F-E  9:C-D  10:D-G
% 11:G-F  12:G-J  13:J-I

% Friction factors (dimensionless, given in problem table)
f = [0.0223; 0.0175; 0.0223; 0.0218; 0.0218; ...
     0.0233; 0.0175; 0.0188; 0.0233; 0.0175; ...
     0.0188; 0.0233; 0.0218];

% Pipe lengths in meters (given in problem table)
L = [150; 125; 125; 150; 250; ...
     75; 125; 75; 75; 125; ...
```

```

75; 125; 150];

% Pipe diameters: convert from mm to m (given in problem table)
D = [225; 200; 225; 300; 300; ...
     225; 200; 150; 225; 200; ...
     150; 225; 300] / 1000;

% Initial flow guesses converted from L/s to m^3/s (divide by 1000)
% Positive Q = flow in the stated positive direction for each pipe
% Sign notes:
% Pipe 4 (I-H): problem states H->I so stored as -200 (opposite of I->H label)
% Pipe 12 (G-J): problem states J->G so stored as -200 (opposite of G->J label)
% Pipe 13 (J-I): problem states I->J so stored as -200 (opposite of J->I label)
Q = [200; 100; 0; -200; 200; ...
     100; 0; 0; 100; 0; ...
     200; -200; -200] / 1000; % units: m^3/s

% Gravitational acceleration
g = 9.81; % m/s^2

% Pipe names for display in output tables
pipe_names = {'A-B','B-E','E-I','I-H','H-A','B-C','C-F','F-E', ...
              'C-D','D-G','G-F','G-J','J-I'};

% Node names for display (A=1, B=2, C=3, D=4, E=5, F=6, G=7, H=8, I=9, J=10)
node_names = {'A','B','C','D','E','F','G','H','I','J'};

%% =====
% SECTION 2: COMPUTE RESISTANCE COEFFICIENTS K
% =====
% Darcy-Weisbach:  $h_L = f \cdot (L/D) \cdot (V^2/2g)$ 
% Substitute  $V = Q/A$  and  $A = \pi \cdot D^2/4$ :
%  $h_L = K \cdot Q^2$  where  $K = 8 \cdot f \cdot L / (g \cdot \pi^2 \cdot D^5)$ 
% For signed Hardy Cross iteration:
%  $h_L = K \cdot |Q| \cdot Q$  (sign preserved for loop direction)

% Cross-sectional area of each pipe ( $m^2$ )
A_pipe = pi .* D.^2 / 4;

% Resistance coefficient K for each pipe ( $s^2/m^5$ )
K = (8 .* f .* L) ./ (g .* pi.^2 .* D.^5);

% Print K values for each pipe
fprintf('=== PIPE PROPERTIES AND RESISTANCE COEFFICIENTS ===\n');
fprintf('%-6s %8s %8s %8s %14s\n','Pipe','f','L(m)','D(mm)','K ( $s^2/m^5$ )');
for i = 1:13
    fprintf('%-6s %8.4f %8.1f %8.1f %14.4f\n', ...
           pipe_names{i}, f(i), L(i), D(i)*1000, K(i));
end
fprintf('\n');

%% =====
% SECTION 3: LOOP DEFINITIONS
% =====
% Each loop lists pipe indices with sign:
% +pipe_index = pipe traversed in its defined positive direction
% -pipe_index = pipe traversed opposite to its defined direction
%
% Pipe positive directions:
% 1:A->B, 2:B->E, 3:E->I, 4:I->H, 5:H->A
% 6:B->C, 7:C->F, 8:F->E, 9:C->D, 10:D->G
% 11:G->F, 12:G->J, 13:J->I
%
% LP1 clockwise path A->B->E->I->H->A:
% A->B = pipe 1 forward (+1)
% B->E = pipe 2 forward (+2)
% E->I = pipe 3 forward (+3)
% I->H = pipe 4 forward (+4)
% H->A = pipe 5 forward (+5)
%
% LP2 clockwise path B->C->F->E->B:
% B->C = pipe 6 forward (+6)
% C->F = pipe 7 forward (+7)
% F->E = pipe 8 forward (+8)
% E->B = pipe 2 reverse (-2)

```

```

%
% LP3 clockwise path C->D->G->F->C:
% C->D = pipe 9 forward (+9)
% D->G = pipe 10 forward (+10)
% G->F = pipe 11 forward (+11)
% F->C = pipe 7 reverse (-7)
%
% LP4 clockwise path E->F->G->J->I->E:
% E->F = pipe 8 reverse (-8) pipe 8 is F->E
% F->G = pipe 11 reverse (-11) pipe 11 is G->F
% G->J = pipe 12 forward (+12)
% J->I = pipe 13 forward (+13)
% I->E = pipe 3 reverse (-3) pipe 3 is E->I

% Store loop definitions as cell array of signed pipe index vectors
loops = {[ 1, 2, 3, 4, 5], ... % LP1: A-B-E-I-H-A
[ 6, 7, 8, -2], ... % LP2: B-C-F-E-B
[ 9, 10, 11, -7], ... % LP3: C-D-G-F-C
[-8,-11, 12, 13, -3]}; % LP4: E-F-G-J-I-E

num_loops = 4; % Total number of independent loops

%%=====
% SECTION 4: PIPE CONNECTIVITY FOR MASS BALANCE CHECK
%=====
% conn(p,:) = [from_node, to_node] for pipe p
% Positive Q means flow goes from_node -> to_node
conn = [1,2; % pipe 1: A->B
2,5; % pipe 2: B->E
5,9; % pipe 3: E->I
9,8; % pipe 4: I->H
8,1; % pipe 5: H->A
2,3; % pipe 6: B->C
3,6; % pipe 7: C->F
6,5; % pipe 8: F->E
3,4; % pipe 9: C->D
4,7; % pipe 10: D->G
7,6; % pipe 11: G->F
7,10; % pipe 12: G->J
10,9]; % pipe 13: J->I

% External flows at nodes (positive = flow INTO the network)
% Node indices: A=1,B=2,C=3,D=4,E=5,F=6,G=7,H=8,I=9,J=10
external = zeros(10,1);
external(8) = +0.400; % Node H: inlet +400 L/s = +0.4 m^3/s
external(5) = -0.100; % Node E: outlet -100 L/s
external(4) = -0.100; % Node D: outlet -100 L/s
external(6) = -0.200; % Node F: outlet -200 L/s

%%=====
% SECTION 5: PRINT INITIAL GUESSES AND CHECK MASS BALANCE
%=====
fprintf('=== INITIAL FLOW GUESSES ===\n');
for i = 1:13
    fprintf('Pipe %-5s: Q_initial = %8.2f L/s\n', pipe_names{i}, Q(i)*1000);
end
fprintf('\n');

% Check mass balance at each node with initial guesses
% Net flow at node n = external(n) + sum(Q arriving) - sum(Q departing)
fprintf('=== INITIAL MASS BALANCE CHECK (should be ~0 at each node) ===\n');
for n = 1:10
    balance = external(n); % Start with external source/sink
    for p = 1:13
        if conn(p,2) == n % Pipe p arrives at node n
            balance = balance + Q(p);
        elseif conn(p,1) == n % Pipe p departs from node n
            balance = balance - Q(p);
        end
    end
    fprintf('Node %s: net flow = %+8.4f m^3/s (%+7.2f L/s)\n', ...
        node_names{n}, balance, balance*1000);
end
fprintf('\n');

```

```

%% =====
% SECTION 6: HARDY CROSS ITERATION (while loop)
% =====
% Hardy Cross algorithm for each iteration:
% Step 1: For each pipe in loop, compute:  $hL = K * |Q| * Q$  (signed)
% Step 2: Sum head losses around loop:  $sum\_hL = sum(hL)$ 
% Step 3: Compute flow correction:  $dQ = -sum\_hL / (2 * sum(K*|Q|))$ 
% Step 4: Update all pipe flows in loop:  $Q = Q + sign * dQ$ 
% Step 5: Immediately move to next loop using updated Q values
% Repeat until BOTH criteria met:
%  $max|sum\_hL| < 0.01 \text{ m}$  AND  $max|dQ| < 0.001 \text{ m}^3/\text{s}$ 

tol_hL = 0.01; % Head loss convergence threshold (m) - from assignment
tol_dQ = 0.001; % Flow correction convergence threshold ( $\text{m}^3/\text{s}$ ) - from assignment

iter = 0; % Iteration counter
converged = false; % Convergence flag

fprintf('==== HARDY CROSS ITERATIONS ====\n');
fprintf('%6s %12s %14s\n','Iter','max|sum_hL|(m)','max|dQ|( $\text{m}^3/\text{s}$ )');

% while loop as suggested by assignment guidelines
while ~converged
    iter = iter + 1; % Increment iteration counter
    max_hL = 0; % Reset max head loss for this iteration
    max_dQ = 0; % Reset max flow correction for this iteration

    % Loop through each of the 4 flow loops sequentially
    % Q values are updated immediately so later loops use updated values
    for k = 1:num_loops
        loop_pipes = loops{k}; % Get signed pipe indices for loop k

        sum_hL = 0; % Sum of head losses around loop k
        sum_denom = 0; % Sum of  $2*K*|Q|$  for denominator of dQ formula

        % Step 1 & 2: Calculate head loss for each pipe in loop k
        for idx = 1:length(loop_pipes)
            p_signed = loop_pipes(idx); % Signed pipe index
            p = abs(p_signed); % Actual pipe number (1-13)
            sign_loop = sign(p_signed); % +1 if forward, -1 if reverse

            % Flow in the loop traversal direction
            Q_loop = sign_loop * Q(p);

            % Head loss in loop traversal direction:  $hL = K*|Q|*Q$ 
            sum_hL = sum_hL + K(p) * abs(Q_loop) * Q_loop;

            % Denominator:  $2 * K * |Q|$  (always positive)
            sum_denom = sum_denom + 2 * K(p) * abs(Q_loop);
        end

        % Step 3: Compute flow correction dQ for loop k
        if sum_denom > 0
            dQ = -sum_hL / sum_denom; % Hardy Cross correction formula
        else
            dQ = 0; % Avoid divide-by-zero if all flows are zero
        end

        % Step 4: Update pipe flows for all pipes in loop k
        % Pipes shared between loops are updated here immediately
        for idx = 1:length(loop_pipes)
            p_signed = loop_pipes(idx);
            p = abs(p_signed);
            sign_loop = sign(p_signed);
            % Add correction in loop direction; shared pipes get updated
            Q(p) = Q(p) + sign_loop * dQ;
        end

        % Track the worst-case values across all loops this iteration
        max_hL = max(max_hL, abs(sum_hL));
        max_dQ = max(max_dQ, abs(dQ));
    end

    % Print every iteration so progress is visible
    fprintf('%6d %14.6f %14.6f\n', iter, max_hL, max_dQ);
end

```



```

% Check both convergence criteria simultaneously
if max_hL < tol_hL && max_dQ < tol_dQ
    converged = true;
    fprintf('---> CONVERGED at iteration %d\n', iter);
end

% Safety stop to prevent infinite loop
if iter >= 10000
    fprintf('WARNING: Maximum iterations reached without convergence!\n');
    break;
end
end

% Stop timer and record elapsed time
elapsed = toc;
fprintf('\nTotal code runtime: %.6f seconds\n', elapsed);
fprintf('Total Hardy Cross iterations to converge: %d\n\n', iter);

%% =====
% SECTION 7: FINAL FLOW RESULTS TABLE
% =====
fprintf('=== FINAL FLOW DISTRIBUTION (after convergence) ===\n');
fprintf('%-6s %12s %10s %s\n','Pipe','Q (m^3/s)','Q (L/s)','Flow Direction');
fprintf('%s\n', repmat('-',1,65));
for i = 1:13
    % Determine actual flow direction based on sign of Q
    if Q(i) >= 0
        dir_str = sprintf('%s --> %s', node_names{conn(i,1)}, node_names{conn(i,2)});
    else
        dir_str = sprintf('%s --> %s (reversed)', node_names{conn(i,2)}, node_names{conn(i,1)});
    end
    fprintf('%-6s %12.6f %10.4f %s\n', pipe_names{i}, Q(i), Q(i)*1000, dir_str);
end
fprintf('%s\n\n', repmat('=',1,65));

%% =====
% SECTION 8: VERIFY FINAL MASS BALANCE AT EVERY NODE
% =====
% Assignment requires final solution satisfies mass conservation
fprintf('=== FINAL MASS BALANCE VERIFICATION (should be ~0 at each node) ===\n');
all_ok = true;
for n = 1:10
    balance = external(n);
    for p = 1:13
        if conn(p,2) == n
            balance = balance + Q(p); % Flow arriving at node n
        elseif conn(p,1) == n
            balance = balance - Q(p); % Flow departing from node n
        end
    end
    status = '';
    if abs(balance) > 1e-6
        status = ' *** DOES NOT BALANCE - CHECK ***';
        all_ok = false;
    end
    fprintf('Node %s: net flow = %+10.6f m^3/s (%+8.4f L/s)%s\n', ...
        node_names{n}, balance, balance*1000, status);
end
if all_ok
    fprintf('All nodes satisfy mass conservation. Solution is valid.\n');
end

%% =====
% SECTION 9: VERIFY FINAL HEAD LOSS BALANCE IN EACH LOOP
% =====
% Sum of head losses around each loop should be ~0 at convergence
fprintf('\n=== FINAL HEAD LOSS BALANCE PER LOOP (should be ~0) ===\n');
for k = 1:num_loops
    loop_pipes = loops{k};
    sum_hL_final = 0;
    for idx = 1:length(loop_pipes)
        p_signed = loop_pipes(idx);
        p = abs(p_signed);
        sign_loop = sign(p_signed);
    end
end

```

```

    Q_loop = sign_loop * Q(p);
    sum_hL_final = sum_hL_final + K(p) * abs(Q_loop) * Q_loop;
end
fprintf('Loop %d: sum(hL) = %+6f m\n', k, sum_hL_final);
end

%%=====
% SECTION 10: TASK 2 - TIME TO FILL TANK WITH 5 kW PUMP
%=====
% Pump lifts water from reservoir (10m below) into open tank
% Tank diameter D = 20m, height H = 5m (from diagram)
% Outlet valve to pipe system is CLOSED - pump fills tank only
% Pump power P = 5 kW, efficiency = 100%
%
% Energy balance: P = rho * g * Q_pump * h_total
% h_total = h_static + h_friction
% h_static = 10 m (elevation difference)
% h_friction = K_tube * Q^2 (Darcy-Weisbach for supply tube)
%
% Solve iteratively because f depends on Re which depends on Q:
% Q = P / (rho * g * (h_static + K_tube * Q^2))

fprintf('\n==== TASK 2: TIME TO FILL TANK (5 kW pump, outlet valve CLOSED) ===\n');

% Tank geometry from diagram
D_tank = 20; % Tank diameter (m)
H_tank = 5; % Tank height (m)
V_tank = pi/4 * D_tank^2 * H_tank; % Tank volume (m^3)
fprintf('Tank dimensions: D = %.0f m, H = %.0f m\n', D_tank, H_tank);
fprintf('Tank volume = pi/4 * %.0f^2 * %.0f = %.4f m^3\n', D_tank, H_tank, V_tank);

% Supply tube properties (iron pipe, D = 200 mm, from diagram note)
D_tube = 0.200; % Tube inner diameter (m)
L_tube_total = 10 + 20; % Total tube length: 10m vertical + 20m horizontal (m)
epsilon_tube = 0.046e-3; % Iron pipe roughness (m), standard value
A_tube = pi * D_tube^2/4; % Tube cross-sectional area (m^2)

% Fluid properties (water at room temperature)
rho = 1000; % Density (kg/m^3)
mu = 1e-3; % Dynamic viscosity (Pa.s)

% Pump parameters for Task 2
P_pump = 5000; % Pump power = 5 kW (W)
eta = 1.0; % Pump efficiency = 100% (as given)
h_static = 10.0; % Static head = 10 m (reservoir is 10m below tank)

% Iterative solution settings (as required by assignment)
tol_pump = 0.001; % Convergence tolerance for pump iteration (from assignment)
f_tube = 0.02; % Initial guess for friction factor (fully turbulent approximation)
Q_pump_old = 0.01; % Initial guess for pump flow rate (m^3/s) = 10 L/s

fprintf('\nInitial guess: Q_pump = %.4f m^3/s (%.2f L/s)\n', Q_pump_old, Q_pump_old*1000);
fprintf('Initial guess: f_tube = %.4f\n', f_tube);
fprintf('Tolerance for pump iteration: %.4f\n', tol_pump);
fprintf('%6s %12s %10s %10s\n','Iter','Q_pump(m^3/s)','f_tube','Error');

% while loop for pump flow iteration (as required by assignment)
pump_iter = 0; % Pump iteration counter
pump_conv = false; % Pump convergence flag

while ~pump_conv
    pump_iter = pump_iter + 1;

    % Compute Reynolds number based on current Q guess
    Re = rho * Q_pump_old * D_tube / (mu * A_tube);

    % Update friction factor using Colebrook-White equation
    % 1/sqrt(f) = -2*log10(epsilon/(3.7*D) + 2.51/(Re*sqrt(f)))
    f_tube = (-2 * log10(epsilon_tube/(3.7*D_tube) + 2.51/(Re*sqrt(f_tube))))^(-2);

    % Compute tube resistance coefficient K_tube
    K_tube = (8 * f_tube * L_tube_total) / (g * pi^2 * D_tube^5);

    % Fixed-point iteration: rearrange energy balance for Q
    % P = rho*g*Q*(h_static + K_tube*Q^2) => Q = P/(rho*g*(h_static + K_tube*Q_old^2))

```

```

Q_pump_new = P_pump / (rho * g * (h_static + K_tube * Q_pump_old^2));

% Compute error between iterations
err = abs(Q_pump_new - Q_pump_old);

fprintf('%6d %12.6f %10.6f %10.6f\n', pump_iter, Q_pump_new, f_tube, err);

% Update Q for next iteration
Q_pump_old = Q_pump_new;

% Check convergence against tolerance of 0.001 (as required by assignment)
if err < tol_pump
    pump_conv = true;
    fprintf('---> Pump iteration CONVERGED at iteration %d\n', pump_iter);
end

% Safety stop
if pump_iter >= 1000
    fprintf('WARNING: Pump iteration did not converge!\n');
    break;
end
end

% Final pump results for Task 2
Q_pump = Q_pump_old; % Converged pump flow rate
h_f_tube = K_tube * Q_pump^2; % Friction head loss in tube
h_total_pump = h_static + h_f_tube; % Total head the pump delivers
P_check = rho * g * Q_pump * h_total_pump; % Verify power (should be ~5000 W)
t_fill = V_tank / Q_pump; % Time to fill tank (s)

fprintf('\n--- Task 2 Results ---\n');
fprintf('Converged pump flow rate Q = %.6f m^3/s = %.4f L/s\n', Q_pump, Q_pump*1000);
fprintf('Tube friction factor f = %.6f\n', f_tube);
fprintf('Tube resistance coeff K = %.4f s^2/m^5\n', K_tube);
fprintf('Friction head loss in tube = %.4f m\n', h_f_tube);
fprintf('Total head delivered = %.4f m\n', h_total_pump);
fprintf('Power verification = %.2f W (should be ~5000 W)\n', P_check);
fprintf('Tank volume = %.4f m^3\n', V_tank);
fprintf('Time to fill tank = %.2f s\n', t_fill);
fprintf(' = %.2f minutes\n', t_fill/60);
fprintf(' = %.4f hours\n', t_fill/3600);
fprintf('Number of pump iterations = %d\n', pump_iter);

%% =====
% SECTION 11: TASK 3 - IS 5 kW ENOUGH FOR THE PIPE SYSTEM?
% =====
% When the outlet valve is OPEN, pump must supply Q = 400 L/s to the network
% Check if 5 kW provides enough head at Q = 0.4 m^3/s
%
% Required head = h_static + K_tube * Q_system^2
% Required power = rho * g * Q_system * h_required

fprintf('\n=== TASK 3: IS 5 kW PUMP SUFFICIENT FOR PIPE SYSTEM? ===\n');

Q_system = 0.400; % Total flow demand of pipe network (m^3/s) = 400 L/s

% Compute Reynolds number at system flow rate
Re_sys = rho * Q_system * D_tube / (mu * A_tube);
fprintf('System flow rate = %.3f m^3/s (%.0f L/s)\n', Q_system, Q_system*1000);
fprintf('Reynolds number at system flow = %.0f\n', Re_sys);

% Iterate Colebrook to find f at system flow rate
f_sys = 0.02; % Initial guess
it_sys = 0; % Iteration counter for this sub-loop

while true
    it_sys = it_sys + 1;
    f_sys_new = (-2*log10(epsilon_tube/(3.7*D_tube) + 2.51/(Re_sys*sqrt(f_sys))))^(-2);
    if abs(f_sys_new - f_sys) < 1e-10 % Tight tolerance for f convergence
        break;
    end
    f_sys = f_sys_new;
    if it_sys > 1000, break; end
end
f_sys = f_sys_new;

```

```

% Compute system tube resistance and required head
K_tube_sys = (8 * f_sys * L_tube_total) / (g * pi^2 * D_tube^5);
h_f_sys = K_tube_sys * Q_system^2; % Friction head at system flow
h_total_sys = h_static + h_f_sys; % Total required head
P_required = rho * g * Q_system * h_total_sys; % Required pump power (W)

fprintf('Friction factor at system flow = %.6f\n', f_sys);
fprintf('Friction head loss in tube = %.4f m\n', h_f_sys);
fprintf('Total required head = %.4f m\n', h_total_sys);
fprintf('Required pump power = %.2f W = %.4f kW\n', ...
    P_required, P_required/1000);
fprintf('Available pump power = 5.0000 kW\n');

% Conclusion
if P_required > 5000
    P_suggested = P_required * 1.20; % Add 20% safety margin
    fprintf('\nCONCLUSION: 5 kW pump is NOT sufficient for the pipe system.\n');
    fprintf('Minimum required power = %.4f kW\n', P_required/1000);
    fprintf('Suggested power (20%% margin) = %.4f kW\n', P_suggested/1000);
else
    fprintf('\nCONCLUSION: 5 kW pump IS sufficient for the pipe system.\n');
end

%% =====
% SECTION 12: FINAL SUMMARY
% =====
fprintf('\n%s\n', repmat('=',1,65));
fprintf('FINAL SUMMARY\n');
fprintf('%s\n', repmat('=',1,65));
fprintf('Hardy Cross converged in : %d iterations\n', iter);
fprintf('Total code runtime : %.6f seconds\n', elapsed);
fprintf('\nFinal pipe flow rates:\n');
fprintf('%-6s %-10s %-10s %s\n', 'Pipe', 'Q (m^3/s)', 'Q (L/s)', 'Direction');
fprintf('%s\n', repmat('-',1,65));
for i = 1:13
    if Q(i) >= 0
        dir_str = sprintf('%s --> %s', node_names{conn(i,1)}, node_names{conn(i,2)});
    else
        dir_str = sprintf('%s --> %s', node_names{conn(i,2)}, node_names{conn(i,1)});
    end
    fprintf('%-6s %-10.6f %-10.4f %s\n', pipe_names{i}, Q(i), abs(Q(i))*1000, dir_str);
end
fprintf('%s\n', repmat('=',1,65));
fprintf('\nTask 2: Time to fill tank = %.2f s = %.2f min = %.4f hours\n', ...
    t_fill, t_fill/60, t_fill/3600);
if P_required > 5000
    fprintf('Task 3: 5 kW NOT sufficient. Suggested power = %.4f kW\n', P_suggested/1000);
else
    fprintf('Task 3: 5 kW IS sufficient for the pipe system.\n');
end
fprintf('%s\n', repmat('=',1,65));

```

Hardy Cross converged in : 14 iterations

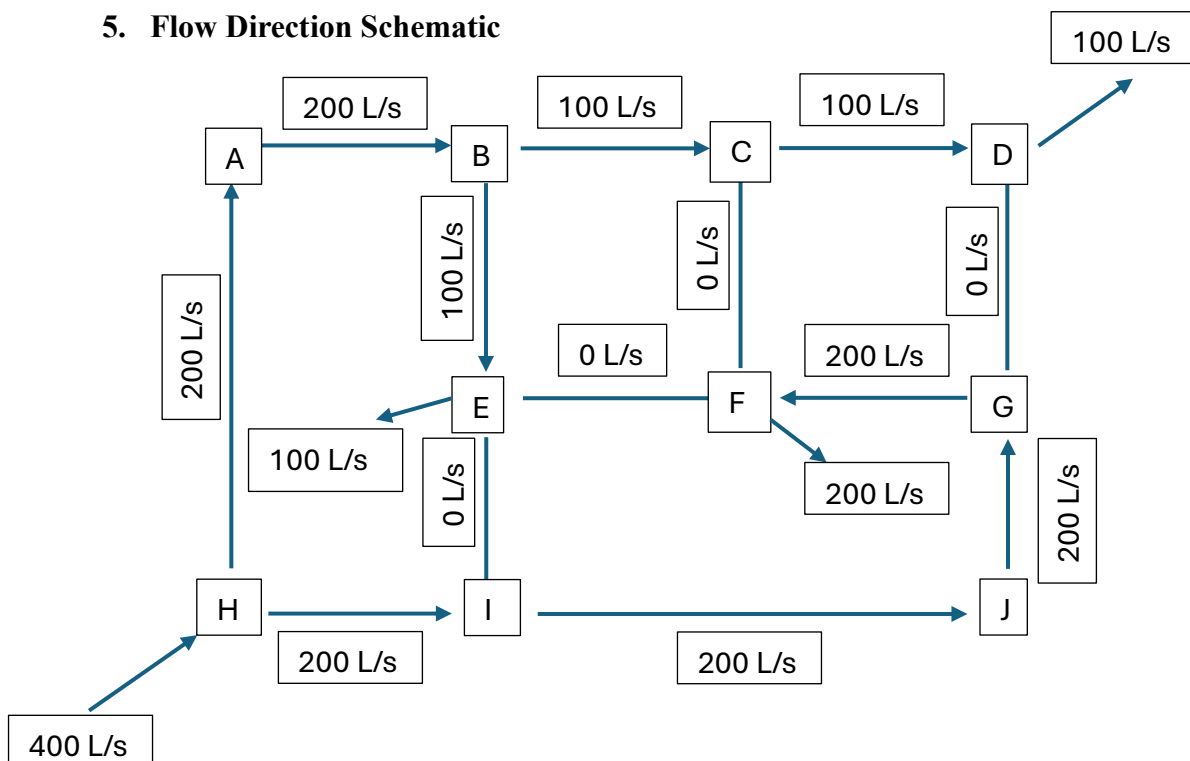
Total code runtime on my PC : 0.011259 seconds

4. Final Solution

Pipe Section	Final Flow Rate (L/s)	Final Flow Rate (m ³ /s)	Direction
A-B	144.3423	0.144342	A to B
B-E	30.1132	0.030113	B to E
E-I	-133.2636	-0.133264	I to E
I-H	-255.6577	-0.255658	H to I
H-A	144.3423	0.144342	H to A
B-C	114.2291	0.114229	B to C
C-F	77.6358	0.077636	C to F
F-E	-63.3768	-0.063377	E to F
C-D	36.5933	0.036593	C to D
D-G	-63.4067	-0.063407	G to D
G-F	58.9874	0.058987	G to F
G-J	-122.3941	-0.122394	J to G
J-I	-122.3941	-0.122394	I to J

Number of Iterations: 14

5. Flow Direction Schematic



6. Pump Selection

Company: Bell & Gossett (a Xylem brand)

Model: Series e-1510, Size 8G

The Bell & Gossett Series e-1510 Size 8G was selected because its large 8-inch discharge and G-frame volute comfortably meets the 400 L/s flow demand determined in Task 3, with motor options up to 150 HP providing sufficient power with a safety margin. The e-1510 features stainless steel impellers designed using CFD technology for maximum efficiency, reducing long-term operating costs. Bell & Gossett's 100+ years of industry experience also ensures reliable performance, readily available spare parts, and strong technical support.

